

DEV_IMPL_GUIDE - FOUNDATION_DB

1. Purpose

This guide explains how to implement the PostgreSQL foundation.

Use it with FOUNDATION_DB.md.

This is a provisioning/runtime-setup guide. It is intentionally written as infrastructure setup steps rather than a business request execution flow.

2. What You Are Building

You are building the database foundation for SOPHIX:

- 6 PostgreSQL nodes retained because DB is critical.
- Three logical clusters: A, B, and C.
- Each cluster has one primary and one replica.
- Backup, monitoring, failover, and access rules.

3. Step 1 - Provision Nodes

Recommended medium layout:

Cluster	Primary	Replica	Purpose
Cluster A	pg-a-primary	pg-a-replica	Core business modules.
Cluster B	pg-b-primary	pg-b-replica	Billing/high-write modules.
Cluster C	pg-c-primary	pg-c-replica	Camunda/infra support databases.

Use Ubuntu Server 24.04 LTS or RHEL 9.

4. Step 2 - Prepare Disks

Recommended partitions:

Mount	Purpose
/	OS.
/var/lib/postgresql	PostgreSQL data.
/var/log/postgresql	DB logs.
/backup	Local backup staging if used.

Use separate fast disks for data and WAL where budget allows.

5. Step 3 - Install PostgreSQL

1. Install approved PostgreSQL version.
2. Pin package version.
3. Configure system limits.
4. Enable service.
5. Set timezone to UTC.
6. Enable pg_stat_statements.

Verification:

```
psql -c "SELECT version();"
psql -c "CREATE EXTENSION IF NOT EXISTS pg_stat_statements;"
```

6. Step 4 - Configure Replication

For each cluster:

1. Create replication user.
2. Enable WAL archiving if used.
3. Configure primary.
4. Initialize replica from primary backup.
5. Start streaming replication.

Verification:

```
SELECT client_addr, state, sync_state
FROM pg_stat_replication;
```

Expected:

```
state = streaming
```

7. Step 5 - Create Databases And Users

Create one database/schema ownership model per module group.

Example:

```
CREATE DATABASE sophix_subscription;
CREATE USER sophix_subscription_app WITH PASSWORD '<secret>';
GRANT CONNECT ON DATABASE sophix_subscription TO sophix_subscription_app;
```

Rules:

- App users must not be superusers.
- Use one application user per module or module group.
- Store passwords in secrets manager.

8. Step 6 - Backup And Restore

Configure:

- Daily full backup or continuous backup.
- WAL archiving.
- Backup encryption.
- Backup retention.
- Restore test schedule.

Minimum restore test:

1. Restore latest backup to isolated environment.
2. Apply WAL to target point.
3. Run smoke queries.
4. Record restore time.

9. Step 7 - Monitoring

Create alerts for:

Metric	Alert
Primary down	Immediate.
Replication lag	Above threshold.
Disk usage	Above 80 percent.
Connection saturation	Above 80 percent.

Metric	Alert
Long transactions	Older than threshold.
Failed backups	Any failure.

10. Step 8 - AWS Option

If using AWS:

- Use Amazon RDS PostgreSQL Multi-AZ.
- Use separate RDS instances/clusters for A, B, C if budget allows.
- Enable automated backups.
- Enable Performance Insights.
- Use Secrets Manager.
- Use private subnets only.

11. Tests

Test:

- App can connect with non-superuser.
- Replica streams from primary.
- Backup restore works.
- Failover runbook works.
- DB rejects unauthorized network access.
- Monitoring alerts fire in test.

12. Common Mistakes

- Letting app users own all databases.
- Skipping restore tests.
- Running Camunda DB on the same overloaded cluster without capacity planning.
- Ignoring replication lag.
- Keeping backups on the same failed disk class only.